nh3pred 0.2.0: vignettes

Getting started

Overview

The package exposes one user-facing function, `predict`. It predicts ammonia emission dynamics following fertilizer application. The model is a recurrent neural network trained on the ALFAM2 database (Hafner et al., 2025). See here for a full description of the model.

This guide covers the three most common workflows:

1. predicting from plot-level data only;
2. supplying a custom time and weather table;
3. comparing several application scenarios.

Import the package

```
import matplotlib.pyplot as plt
import pandas as pd

from nh3pred import predict
```

Minimal prediction

`pdat` contains one row per field application. The `id` column uniquely identifies each application and links it to optional time-dependent data.

Only the variables needed to describe this example are supplied below. All other variables are completed with package defaults.

```
pdat = pd.DataFrame(
    {
        "id": [1],
        "tan_app": [70],
        "app_method": ["bc"],
    }
)

predictions = predict(pdat)
```

When `idat` is omitted, predictions use cumulative times 2, 4, and then four-hour intervals from 4 to 48 h, and twelve-hour intervals from 48 to 240 h. The package uses a repeating default daily temperature and wind profile and assumes no rainfall.

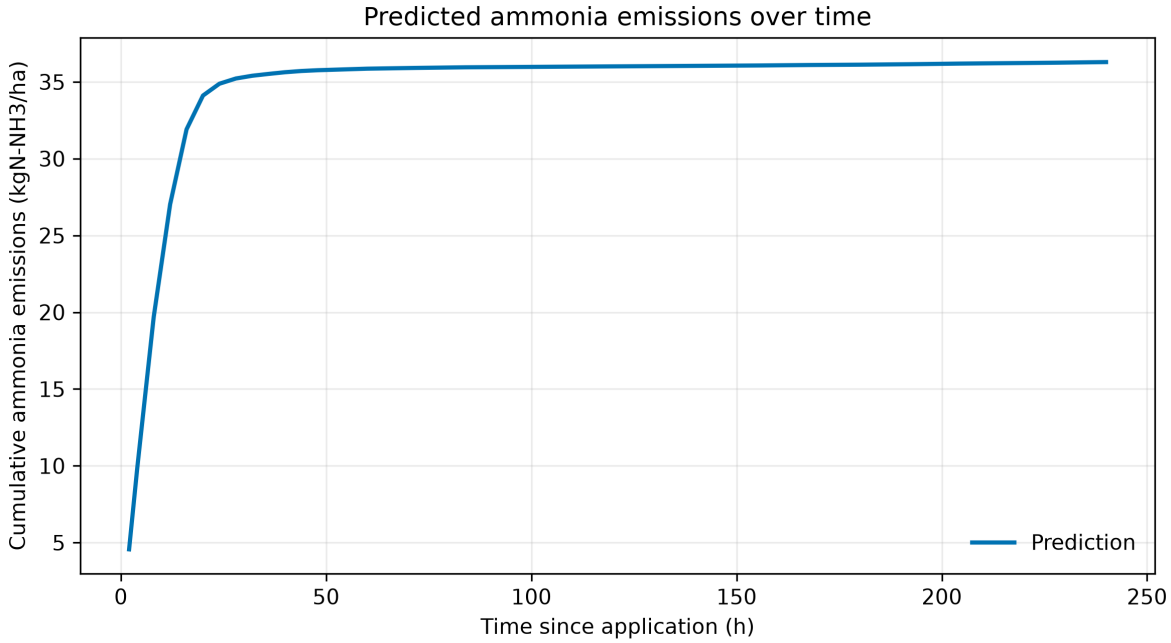
```
predictions[
    [
        "id",
        "ct",
        "air_temp",
        "wind_2m",
        "prediction_erel",
        "prediction_ecum",
    ]
].head()
```

	id	ct	air_temp	wind_2m	prediction_erel	prediction_ecum
0	1	2	16.29	3.31	0.07	4.56
1	1	4	18.54	3.85	0.14	9.88
2	1	8	19.83	4.16	0.28	19.69
3	1	12	16.29	3.31	0.39	27.03
4	1	16	11.46	2.15	0.46	31.91

Plot the prediction

```
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(
    predictions["ct"],
    predictions["prediction_ecum"],
    color="#0072B2",
    linewidth=2,
    label="Prediction",
)
ax.set(
    xlabel="Time since application (h)",
    ylabel="Cumulative ammonia emissions (kgN-NH3/ha)",
    title="Predicted ammonia emissions over time",
)
ax.grid(alpha=0.25)
ax.legend(frameon=False)
```

```
fig.tight_layout()
plt.show()
```



Supply custom time and weather data

`idat` contains one row per application and time point. Time points may be irregular. In this example, air temperature is supplied while wind and rainfall are omitted.

```
idat = pd.DataFrame(
    {
        "id": [1, 1, 1, 1, 1],
        "ct": [2, 8, 20, 32, 48],
        "air_temp": [16, 21, 13, 15, 18],
    }
)

custom_predictions = predict(pdat, idat)

custom_predictions[
    ["id", "ct", "air_temp", "wind_2m", "rain_rate", "prediction_ecum"]
]
```

	id	ct	air_temp	wind_2m	rain_rate	prediction_ecum
0	1	2	16	3.31	0	4.55
1	1	8	21	4.16	0	18.88
2	1	20	13	1.84	0	37.49
3	1	32	15	4.16	0	45.71
4	1	48	18	2.69	0	50.14

Supplied application values are retained unless a documented completion rule applies. All numeric output columns, including times and weather, are rounded to two decimals; calculations use full-precision inputs. A missing `air_temp` or `wind_2m` column is filled by interpolating the corresponding default profile at the requested `ct` values. An absent rainfall column defaults to zero. Missing values inside a supplied weather column are not filled with the default weather profile. Because public predictions represent micrometeorological measurements, `wind_2m` is routed internally to the model's micrometeorological wind input.

Compare several scenarios

Each row of `pdat` may describe a different scenario. Here, three otherwise identical applications differ only in application method.

```
scenario_pdat = pd.DataFrame(
    {
        "id": [1, 2, 3],
        "tan_app": [70, 70, 70],
        "app_method": ["bc", "ts", "os"],
    }
)

scenario_predictions = predict(scenario_pdat)

scenario_predictions.groupby("id", as_index=False).tail(1)[
    ["id", "ct", "app_method", "prediction_ecum"]
]
```

	id	ct	app_method	prediction_ecum
28	1	240	bc	36.29
57	2	240	ts	21.17
86	3	240	os	8.21

See the scenario comparison vignette for a complete visual comparison.

Control computation time

```
predictions = predict(
    pdat,
    idat,
    batch_size=128,
)
```

- `batch_size` controls how many application sequences are processed together. Larger values may improve speed but require more memory.

Next steps

Consult the input reference for accepted columns, default values, and input conventions. Predictions outside the ranges represented in the training data should be interpreted cautiously.

Input reference

Application-level inputs

Call `predict(pdat, idat=None, batch_size=128)` with a pandas DataFrame containing one application per `id`. Required columns are `id`, `tan_app` (applied total ammoniacal nitrogen, kg N/ha), and `app_method`. Use positive applied TAN. Identifiers must be consistent between `pdat` and `idat`.

The following tables are generated from the parameters shipped with the installed release. Access to these internal objects is used for documentation only; the public interface is `from nh3pred import predict`.

Input	Accepted categories (case-sensitive)	Default
`app_method`	`bc`, `bsth`, `cs`, `os`, `pi`, `ts`	required
`incorp`	`deep`, `none`, `shallow`	none
`till`	`no`, `unknown`, `yes`	no
`fer_origin`	`cat`, `conc`, `mix`, `other`, `pig`	cat
`fer_forme`	`liquid`, `solid`	liquid
`fer_trt1`	`Acidification`, `Anaerobic digestion`, `Diluted`, `Irrigated`, `none`	none
`fer_trt2`	`Separation`, `none`	none
`crop`	`bare soil`, `cereal`, `grass`, `maize`, `other`, `rapeseed`, `stubble`	grass
`soil_type`	`Calcaric Regosol`, `Haplic Luvisol`, `clay`, `clay.loam`, `loam`, `loamy.sand`, `peat`, `sand`, `sandy.clay.loam`, `sandy.loam`, `silty.clay.loam`, `silty.loam`, `unknown`	clay

Application methods are broadcast (`bc`), band spreading/trailing hose (`bsth`), trailing shoe (`ts`), open slot (`os`), closed slot (`cs`) and pressurised injection (`pi`). Origins include cattle (`cat`), pig (`pig`), concentrate (`conc`), mixed (`mix`) and other (`other`). Treatments are represented by two separate columns: `fer_trt1` and `fer_trt2`. `meas_tech` and `fer_trt3` are internal model variables and are rejected as public inputs.

Numeric input	Unit	Default or missing-value rule
`app_rate`	t/ha	20.457020689428806 + 0.25233812374128917 × `tan_app`
`fer_dm`	%	6.59
`fer_ph`	pH	7.3
`crop_z`	cm	10 for a crop; not applicable for bare soil
`time_incorp`	h	0.05 for incorporation; not applicable for `none`
`furrow_z`	cm	Missing unless supplied
`soil_water`	volumetric fraction	0.2715
`soil_ph`	pH	6.5
`soil_dens`	g/cm³	1.14
`soil_oc`	%	3.49

Supply measured values where possible. Defaults are not substitutes for site characterisation. Selecting a solid fertilizer does not automatically adjust dry matter or produce a low-dry-matter warning. The package does not validate every physical range or scientifically plausible combination of predictors.

Time and weather

Optional `idat` has one row per time point and application, with `id` and `ct` (hours since application). Supply strictly increasing positive times for each application, without duplicates. Weather inputs are `air_temp` (°C), `wind_2m` (m/s at 2 m), and `rain_rate` (mm/h).

If `idat` is omitted, predictions use 29 default time points: 2, 4, then every 4 hours through 48 h, followed by every 12 hours through 240 h. Missing weather columns receive the bundled default profiles for air temperature and wind, and zero rain. Profiles are linearly interpolated at the requested times; outside their time range, endpoint values are held constant. Missing entries in an existing weather column retain the model's missing-value representation. `rain_cum` is computed from interval durations and `rain_rate`; a supplied `rain_cum` column is overwritten.

```
import pandas as pd
from nh3pred import predict

reference_pdat = pd.DataFrame({"id": [1], "tan_app": [100.0], "app_method": ["bc"]})
reference_idat = pd.DataFrame({
    "id": [1, 1, 1], "ct": [2.0, 24.0, 48.0],
    "air_temp": [12.0, 15.0, 13.0], "wind_2m": [2.0, 3.0, 2.0],
```

```
    "rain_rate": [0.0, 0.0, 0.0],
})
predict(reference_pdat, reference_idat)[["id", "ct", "prediction_ecum", "prediction_erel"]]
```

	id	ct	prediction_ecum	prediction_erel
0	1	2.0	6.35	0.06
1	1	24.0	53.77	0.54
2	1	48.0	67.70	0.68

Output and interpretation

`prediction_ecum` is cumulative NH_3 emission in kg N/ha; `prediction_erel` is the fraction of applied TAN emitted (not a percentage). All numeric output columns, including copied inputs, are rounded to two decimals. Computation uses unrounded inputs. The reservoir model constrains cumulative emissions to be non-decreasing and at most the applied TAN. Long-horizon predictions and unusual input combinations remain extrapolations, not validated field forecasts.

Scenario comparison

Objective

This section illustrates how the model can be used to compare six application strategies across a common set of fertiliser scenarios. The strategies are broadcast application (`bc`), broadcast application followed by immediate deep incorporation, band spreading with trailing hoses (`bsth`), trailing shoe (`ts`), open-slot injection (`os`), and closed-slot injection (`cs`).

Setup

```
from itertools import product
import matplotlib.pyplot as plt
from matplotlib.ticker import PercentFormatter
import pandas as pd

from nh3pred import predict
```

Define the scenarios

The scenarios are defined by combining different values of fertiliser origin, application rate and applied TAN, dry matter, and fertiliser pH. The numerical values below are fixed illustrative levels retained from the research scenarios; this example does not compute quantiles from a local training dataset. For `tan_app` and `app_rate`, values at the same quantile level are paired. The full factorial design combines:

- cattle and pig fertiliser origins;
- three paired levels of applied TAN and application rate;
- three levels of dry matter and fertiliser pH;
- the six application strategies.

This gives 324 predictions: 54 fertiliser scenarios for each strategy.

All other inputs, including crop, tillage, and soil type, are omitted and completed with the package defaults.

```
quantile_levels = [0.25, 0.5, 0.75]
quantiles = pd.DataFrame(
    {
        "tan_app": [43.68, 68.59, 106.20],
```

```
        "app_rate": [26.67, 35.00, 45.74],
        "fer_dm": [3.4, 5.5, 7.4],
        "fer_ph": [7.10, 7.35, 7.70],
    },
    index=quantile_levels,
)

method_settings = [
    ("bc", "bc", "none", None),
    ("bc + deep\nincorp.", "bc", "deep", 0.05),
    ("bsth", "bsth", "none", None),
    ("ts", "ts", "none", None),
    ("os", "os", "none", None),
    ("cs", "cs", "none", None),
]

rows = []
for fer_origin, pair_q, dm_q, ph_q, method in product(
    ["cat", "pig"],
    quantile_levels,
    quantile_levels,
    quantile_levels,
    method_settings,
):
    method_label, app_method, incorp, time_incorp = method
    rows.append(
        {
            "fer_origin": fer_origin,
            "tan_app": quantiles.loc[pair_q, "tan_app"],
            "app_rate": quantiles.loc[pair_q, "app_rate"],
            "fer_dm": quantiles.loc[dm_q, "fer_dm"],
            "fer_ph": quantiles.loc[ph_q, "fer_ph"],
            "app_method": app_method,
            "incorp": incorp,
            "time_incorp": time_incorp,
            "method": method_label,
            "tan_app_rate_quantile": pair_q,
            "fer_dm_quantile": dm_q,
            "fer_ph_quantile": ph_q,
        }
    )
```

```
pdat = pd.DataFrame(rows)
pdat.insert(0, "id", range(1, len(pdat) + 1))
pdat.iloc[:,:8].head()
```

	id	fer_origin	tan_app	app_rate	fer_dm	fer_ph	app_method	incorp
0	1	cat	43.68	26.67	3.4	7.1	bc	none
1	2	cat	43.68	26.67	3.4	7.1	bc	deep
2	3	cat	43.68	26.67	3.4	7.1	bsth	none
3	4	cat	43.68	26.67	3.4	7.1	ts	none
4	5	cat	43.68	26.67	3.4	7.1	os	none

Generate predictions

No time-dependent data are supplied, predictions are made using the default time sequence and weather profiles. The output column `prediction_erel` is the predicted cumulative emission divided by the applied TAN.

```
predictions = predict(pdat)

predictions[
    [
        "id",
        "ct",
        "app_method",
        "incorp",
        "prediction_ecum",
        "prediction_erel",
    ]
].head()
```

	id	ct	app_method	incorp	prediction_ecum	prediction_erel
0	1	2	bc	none	2.76	0.06
1	1	4	bc	none	5.74	0.13
2	1	8	bc	none	10.62	0.24
3	1	12	bc	none	13.77	0.32
4	1	16	bc	none	15.68	0.36

Compare relative emissions at 240h

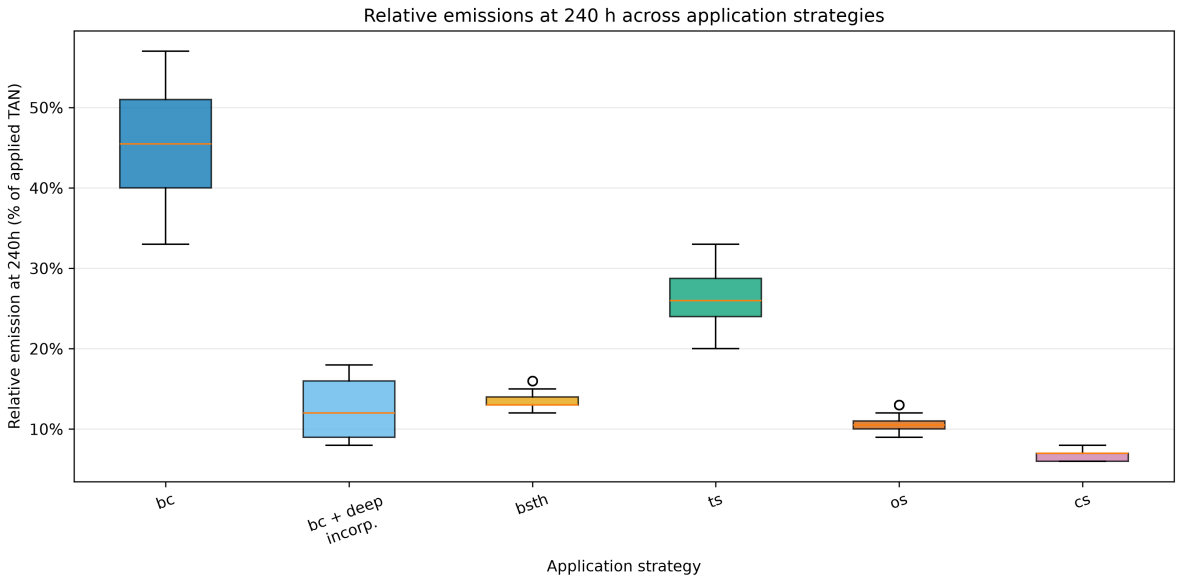
For each scenario, only the prediction at 240 hours is retained. Each box therefore summarises the same 54 combinations of fertiliser origin and numerical predictor values used to define the scenarios.

```
cumulative_predictions = (
    predictions.sort_values("ct")
    .groupby("id", as_index=False)
    .tail(1)
)

method_order = [setting[0] for setting in method_settings]
colors = ["#0072B2", "#56B4E9", "#E69F00", "#009E73", "#D55E00", "#CC79A7"]


def plot_cumulative_emissions(cumulative_predictions, title):
    boxplot_data = [
        cumulative_predictions.loc[
            cumulative_predictions["method"].eq(method), "prediction_erel"
        ]
        for method in method_order
    ]
    fig, ax = plt.subplots(figsize=(11, 5.5))
    boxes = ax.boxplot(
        boxplot_data,
        tick_labels=method_order,
        patch_artist=True,
        showfliers=True,
    )
    for box, color in zip(boxes["boxes"], colors):
        box.set_facecolor(color)
        box.set_alpha(0.75)
    ax.yaxis.set_major_formatter(PercentFormatter(xmax=1))
    ax.set(
        xlabel="Application strategy",
        ylabel="Relative emission at 240h (% of applied TAN)",
        title=title,
    )
    ax.tick_params(axis="x", rotation=20)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    plt.show()
```

```
plot_cumulative_emissions(
    cumulative_predictions,
    "Relative emissions at 240 h across application strategies",
)
```



Compare emission dynamics

The panels below show all scenario trajectories for each application strategy. Thin lines represent individual combinations of fertiliser origin and numerical predictor values, while thick lines show the median trajectory for cattle and pig fertilisers.

```
def plot_emission_dynamics(predictions):
    origins = {
        "cat": ("Cattle", "#0072B2"),
        "pig": ("Pig", "#D55E00"),
    }
    fig, axes = plt.subplots(
        2,
        3,
        figsize=(13, 8),
        sharex=True,
        sharey=True,
```

```
)

for ax, method in zip(axes.flat, method_order):
    method_data = predictions.loc[predictions["method"].eq(method)]
    for origin, (label, color) in origins.items():
        origin_data = method_data.loc[
            method_data["fer_origin"].eq(origin)
        ]
        for _, trajectory in origin_data.groupby("id", sort=False):
            trajectory = trajectory.sort_values("ct")
            ax.plot(
                trajectory["ct"],
                trajectory["prediction_erel"],
                color=color,
                alpha=0.12,
                linewidth=0.8,
            )

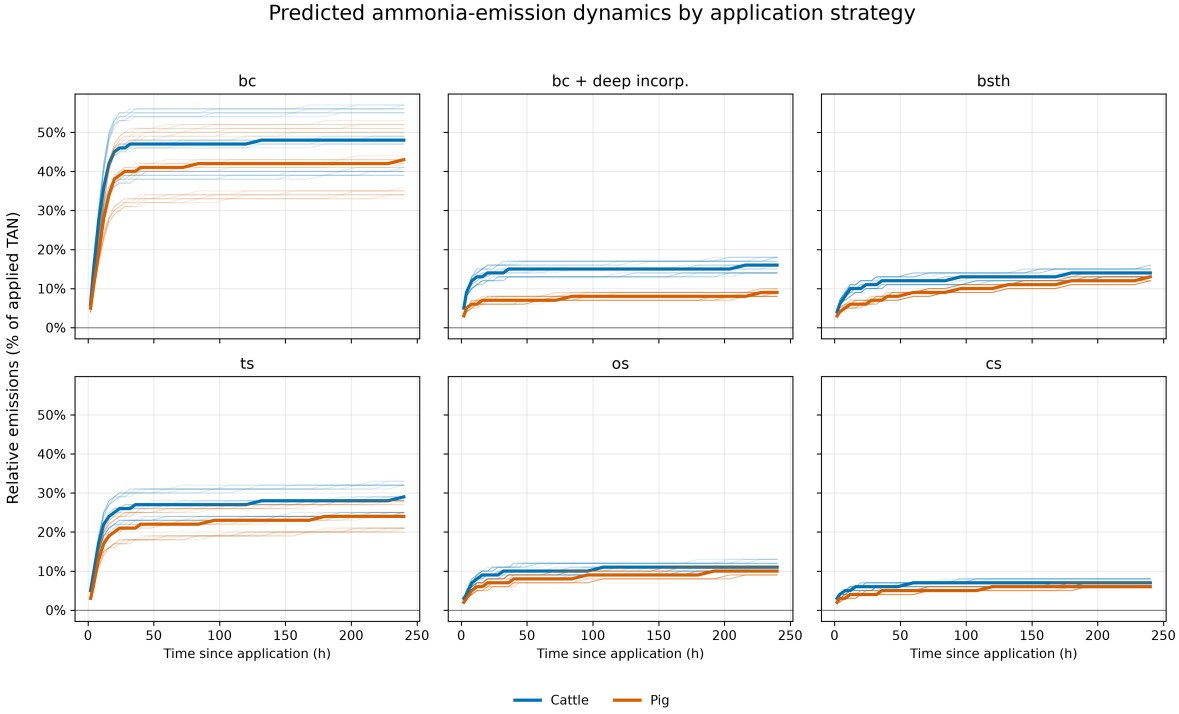
        median = origin_data.groupby("ct", as_index=False)[
            "prediction_erel"
        ].median()
        ax.plot(
            median["ct"],
            median["prediction_erel"],
            color=color,
            linewidth=2.5,
            label=label,
        )

    ax.set_title(method.replace("\n", " "))
    ax.axhline(0, color="black", linewidth=0.8, alpha=0.5)
    ax.yaxis.set_major_formatter(PercentFormatter(xmax=1))
    ax.grid(alpha=0.25)

for ax in axes[-1, :]:
    ax.set_xlabel("Time since application (h)")
fig.supylabel(
    "Relative emissions (% of applied TAN)",
    x=0.01,
)
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(
```

```
        handles,
        labels,
        loc="lower center",
        ncols=2,
        frameon=False,
        bbox_to_anchor=(0.5, 0.01),
    )
    fig.suptitle(
        "Predicted ammonia-emission dynamics by application strategy",
        fontsize=16,
        y=0.98,
    )
    fig.tight_layout(rect=[0, 0.07, 1, 0.95])
    plt.show()


plot_emission_dynamics(predictions)
```



See the getting-started guide for custom weather data and the input reference for accepted variables and categories.

Model and interpretation

A recurrent model coupled to a TAN reservoir

The package predicts the dynamics of ammonia emissions following fertilizer application. Its recurrent neural network describes how environmental and management conditions change the rate of emission. A reservoir decoder converts this rate into a non-decreasing cumulative emission bounded by the applied TAN. In addition to cumulative emissions, the model internally produces an interval-average ammonia flux. This flux is used during training as an additional supervised target, but only cumulative emissions are returned by the package's `predict()` function. Figure 1 summarizes the prediction pathway, from the inputs to cumulative emissions and interval-average fluxes.

Neural architecture

At each time interval, normalized numerical predictors are concatenated with learned embeddings of the categorical predictors. Plot-level characteristics are repeated along the sequence; time and weather inputs change between intervals.

Component	Model configuration
Embedding dimensions per category	8
Recurrent layers	1
Hidden units	512
Recurrent activation	relu
Prediction head	Linear (512 to 6), ReLU, linear (6 to 1)

The recurrent hidden state retains information from preceding intervals. For each interval starting at time t, the final linear layer produces one unrestricted value z_t.

Reservoir decoder

The neural output is transformed into a non-negative kinetic rate:

$$r_t = 0.01 \ \mathrm{softplus}(z_t), \qquad \mathrm{softplus}(z) = \log(1 + \exp(z)).$$

The kinetic rate r_t has units of h^{-1}. Let E_t denote the relative cumulative ammonia emission at time t, expressed as a fraction of applied TAN, and let dt be the duration of interval $[t, t + dt]$. The relative emission during this interval is

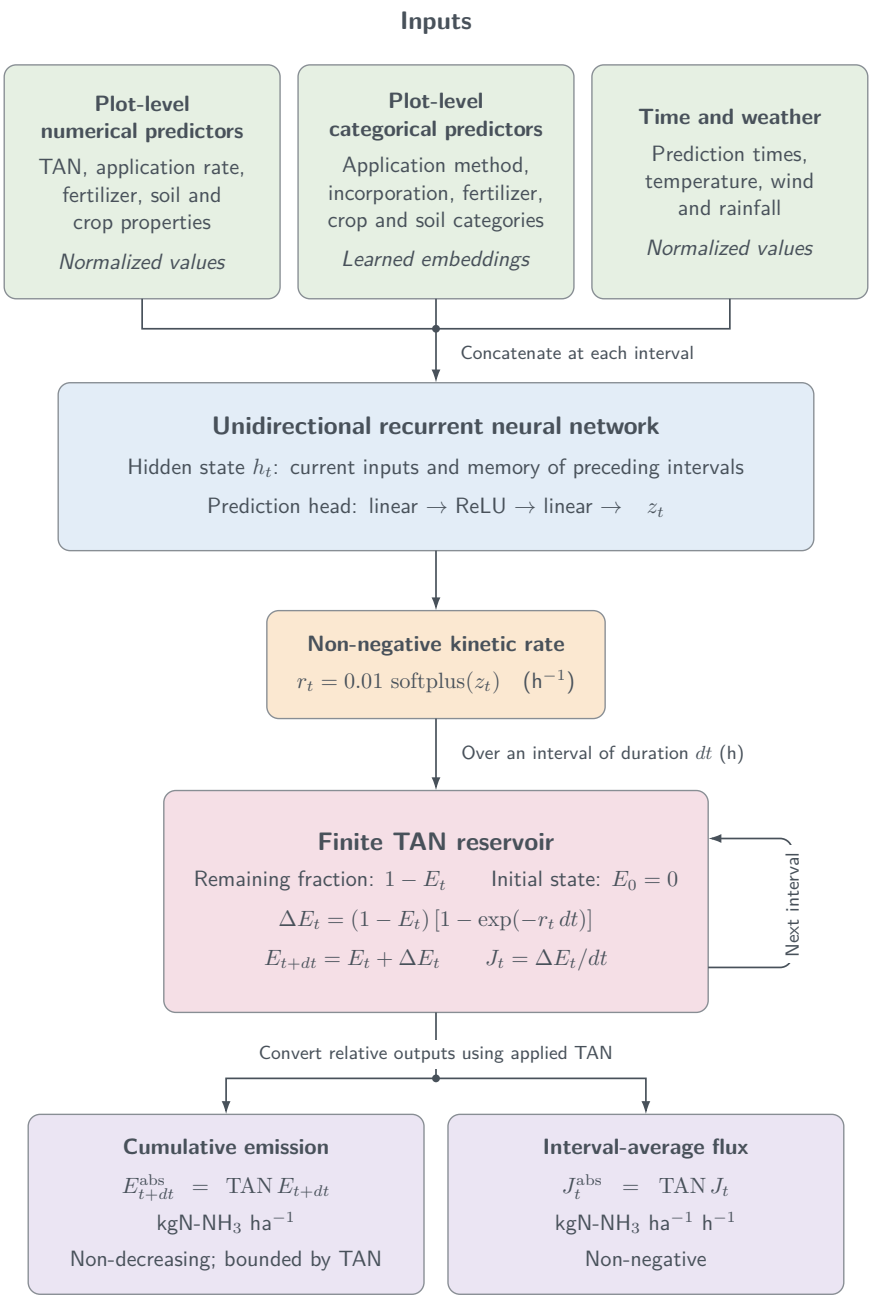


Figure 1: Architecture of the recurrent model coupled to a finite TAN reservoir. The neural network predicts a non-negative kinetic rate, which controls the fraction emitted from the remaining reservoir during each interval. Multiplication by applied TAN gives absolute cumulative emissions and interval-average fluxes.

$$\Delta E_t = (1 - E_t)\,[1 - \exp(-r_t\,dt)]\,.$$

The relative cumulative emission and relative interval-average flux are then

$$E_{t+dt} = E_t + \Delta E_t, \qquad J_t = \frac{\Delta E_t}{dt}.$$

The term $1 - E_t$ represents the fraction of applied TAN remaining in the reservoir. As emissions accumulate, this remaining fraction decreases and limits the amount that can be emitted during subsequent intervals. Starting from $E_0 = 0$, this construction ensures that

$$\Delta E_t \geq 0, \qquad J_t \geq 0, \qquad 0 \leq E_t \leq 1.$$

Consequently, relative cumulative emissions are non-decreasing and remain bounded by 100% of applied TAN (finite precision and output rounding can produce exactly 100%). The corresponding absolute cumulative emission and absolute interval-average flux are

$$E_t^{\text{abs}} = \text{TAN}\,E_t, \qquad J_t^{\text{abs}} = \text{TAN}\,J_t,$$

expressed in kgN-NH_3 ha^{-1} and kgN-NH_3 ha^{-1} h^{-1}, respectively.

Supervision

The model is trained with two targets: cumulative emissions relative to TAN, and the logarithm of interval flux relative to TAN. The loss is the sum of the mean squared errors of these two normalized targets, with flux weight 1. The relative flux is floored at 10^{-6} before taking its logarithm for numerical stability; this floor is not a lower bound imposed on the returned physical flux.

Population and response used for interpretation

The interpretation figures below were computed on all 2,216 prepared applications (including the validation subset), not on an independent test set. They are distributed as precomputed figures with their calculation metadata; rendering this vignette does not retrain the model or read experimental records. They concern the cumulative emission at the last observed time of each plot, expressed in kgN-NH_3 ha^{-1}.

The interpretation dataset contains 2,216 plots from all measurement methods.

Variable importance for the final cumulative emission

Permutation importance for final cumulative emissions is shown in Figure 2. Importance is measured as the increase in MAE after permuting an input across plots, with five repetitions and a fixed random seed. The graph shows the mean increase and the standard deviation across permutations. This follows the permutation model-reliance principle of Fisher, Rudin and Dominici (2019).

Permutations respect complete predictor groups where required:

- Prediction times (`ct`) are treated as a single time-grid feature. During permutation, the time grid from another plot is assigned to the current plot and adapted to match its number of observations while retaining the final prediction horizon of the permuted grid. The weather sequence is then aligned with this modified time grid. The resulting importance therefore reflects the model's sensitivity to both the temporal sampling pattern and the final prediction horizon.
- Weather profiles are transferred between plots while preserving their temporal pattern and adapting them to the target plot's time grid.
- Incorporation type and incorporation delay are permuted together, as are crop type and crop height.

Permutation can still create unusual combinations of the remaining predictors, and correlated variables can share predictive information. A small importance does not establish that a variable is physically not important.

Accumulated local effects for continuous variables

Figure 3 shows the accumulated local effects (ALE) of the continuous predictors on predicted final cumulative emissions. ALE describes how the model prediction changes locally as a predictor varies over the range represented in the interpretation dataset, while limiting the extrapolation associated with changing that predictor globally Apley and Zhu (2020). For each predictor, the observed range is divided into up to ten quantile-based intervals. Within each interval, predictions are compared after moving the predictor to the lower and upper interval boundaries while retaining the other predictor values. These local differences are averaged, accumulated across intervals, and centered around zero. A detailed description of the calculation is provided in the appendix below.

An ALE value of $+2$ kgN-NH_3 ha^{-1} is a centered model effect, not an absolute emission prediction. These curves show local sensitivity over the observed predictor distribution. They reduce the extrapolation involved in a global PDP, without guaranteeing that all perturbed combinations are experimentally plausible.

Crop height is analyzed only where `crop != bare soil`, and incorporation delay only where `incorp != none`.

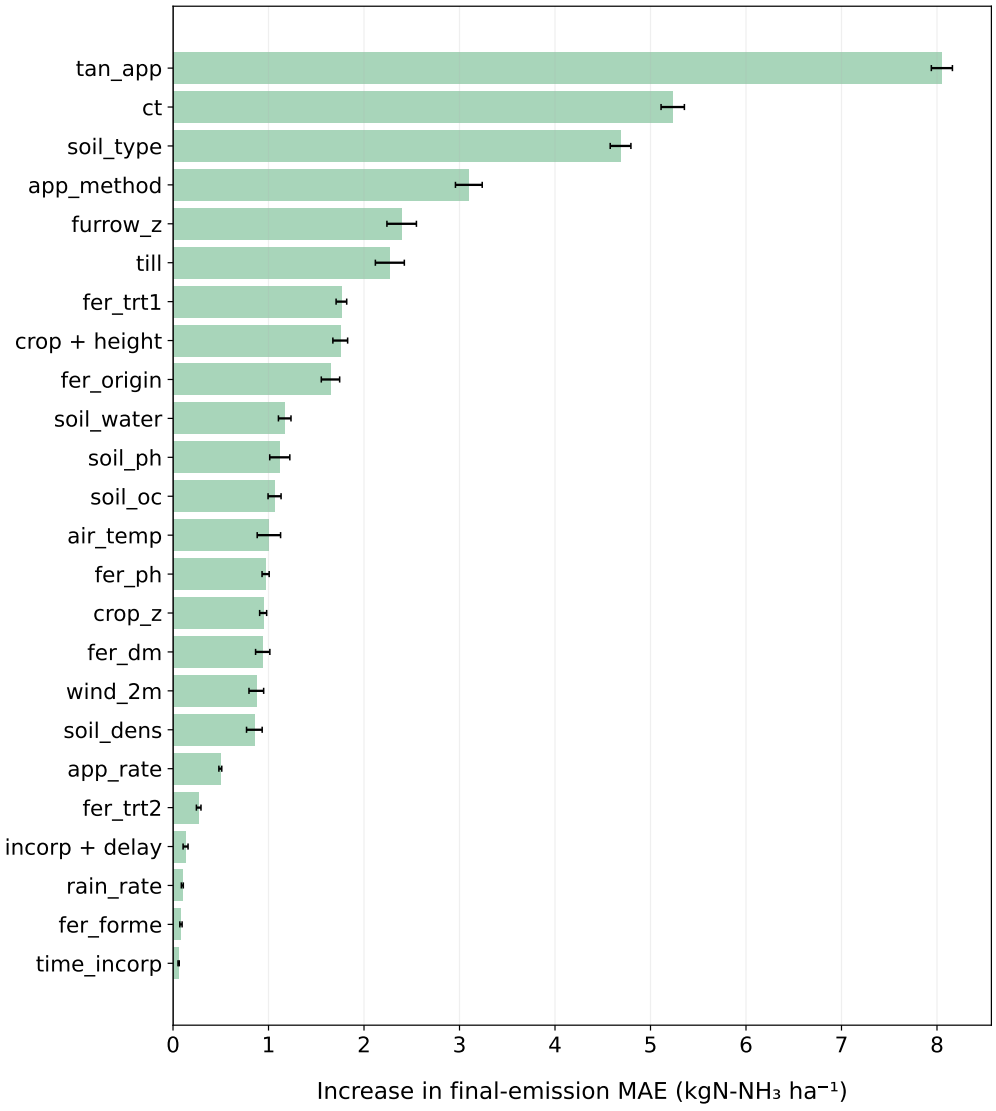


Figure 2: Permutation importance for cumulative emissions. Bars show mean MAE increases and whiskers show permutation standard deviations.

For weather variables, the horizontal axis summarizes the weather conditions over the full trajectory:

- Temperature: interval-duration-weighted mean, modified by an additive shift of the full temperature profile.
- Wind: interval-duration-weighted mean, modified by multiplying the full profile by a positive factor. This ALE is calculated only from trials measured with micrometeorological methods.
- Rainfall: interval-duration-weighted mean rainfall rate, modified by multiplying the full rainfall profile by a positive factor. Dry profiles are excluded.

Partial dependence for categorical variables

Figure 4 shows the partial dependence of final cumulative emissions on the categorical predictors.

For each category observed in the interpretation dataset, that category is assigned to every plot and the predicted final cumulative emissions are averaged.

Most inputs remain fixed. Two dependent-input adjustments are needed:

- For `incorp = none`, the delay becomes non-applicable. For an active incorporation category, an existing observed delay is retained; otherwise the median observed applicable delay is assigned.
- For `crop = bare soil`, height becomes non-applicable. When moving a bare-soil trajectory to vegetation, the median observed applicable height is assigned.

The replacement delay is 0.05 h and the replacement height is 10 cm.

Other dependencies, such as fertilizer treatment and pH, are not reconstructed.

Appendix: Detailed ALE calculation

Let $f(x_i)$ denote the final absolute emission predicted for plot i, and let x_{ij} be the value of continuous predictor j for that plot. The observed values of predictor j are divided into up to ten quantile-based intervals,

$$[z_0, z_1], [z_1, z_2], \ldots, [z_{K-1}, z_K].$$

Because the interval limits are based on quantiles, the intervals are intended to contain similar numbers of plots. However, repeated identical predictor values can cause several quantile boundaries to coincide, reducing the number of distinct intervals and leading to unequal numbers of contributing plots across intervals.

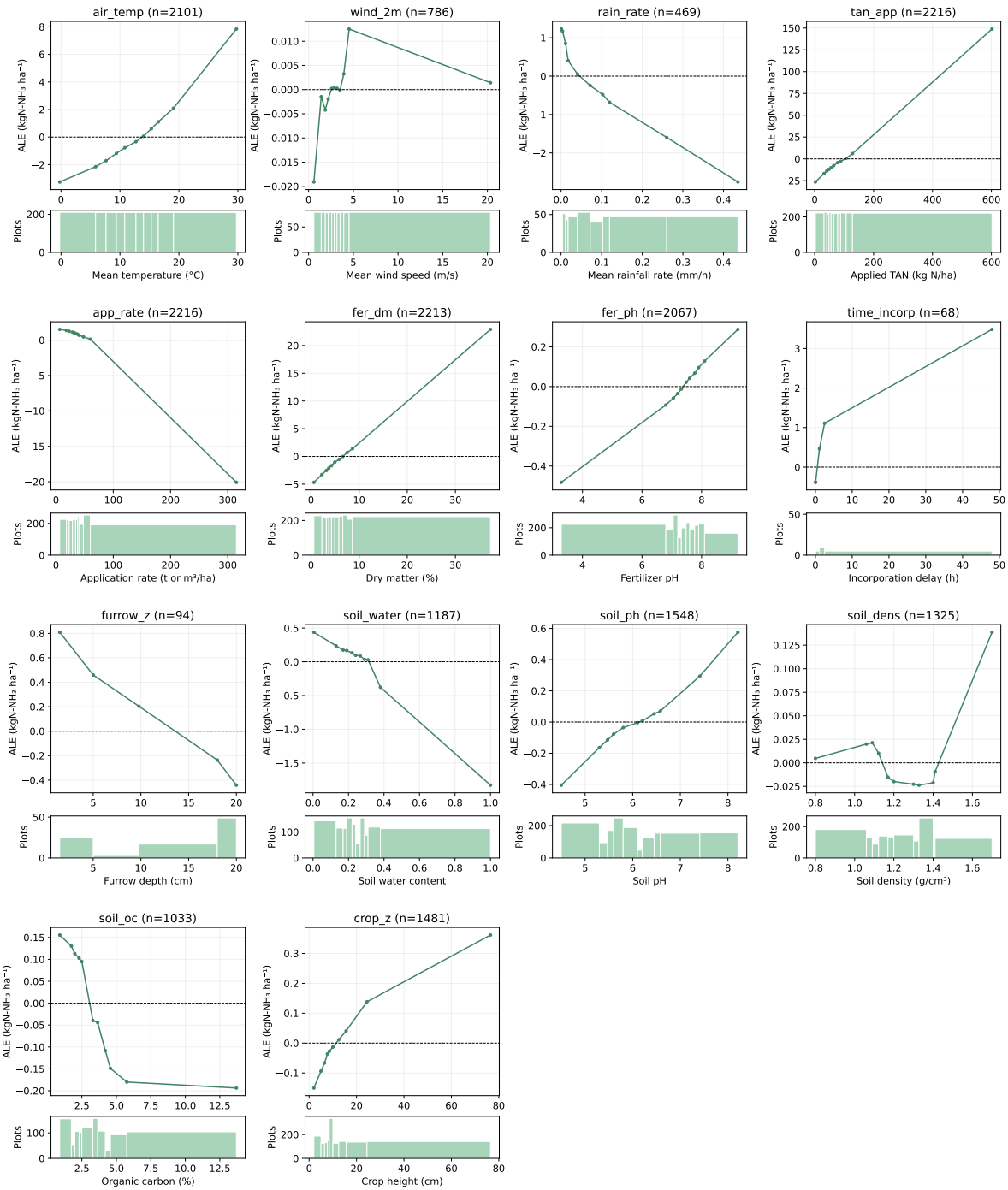


Figure 3: First-order ALE of cumulative emissions. The histograms below each ALE curve show the distribution of the predictor values among the plots contributing to the calculation.

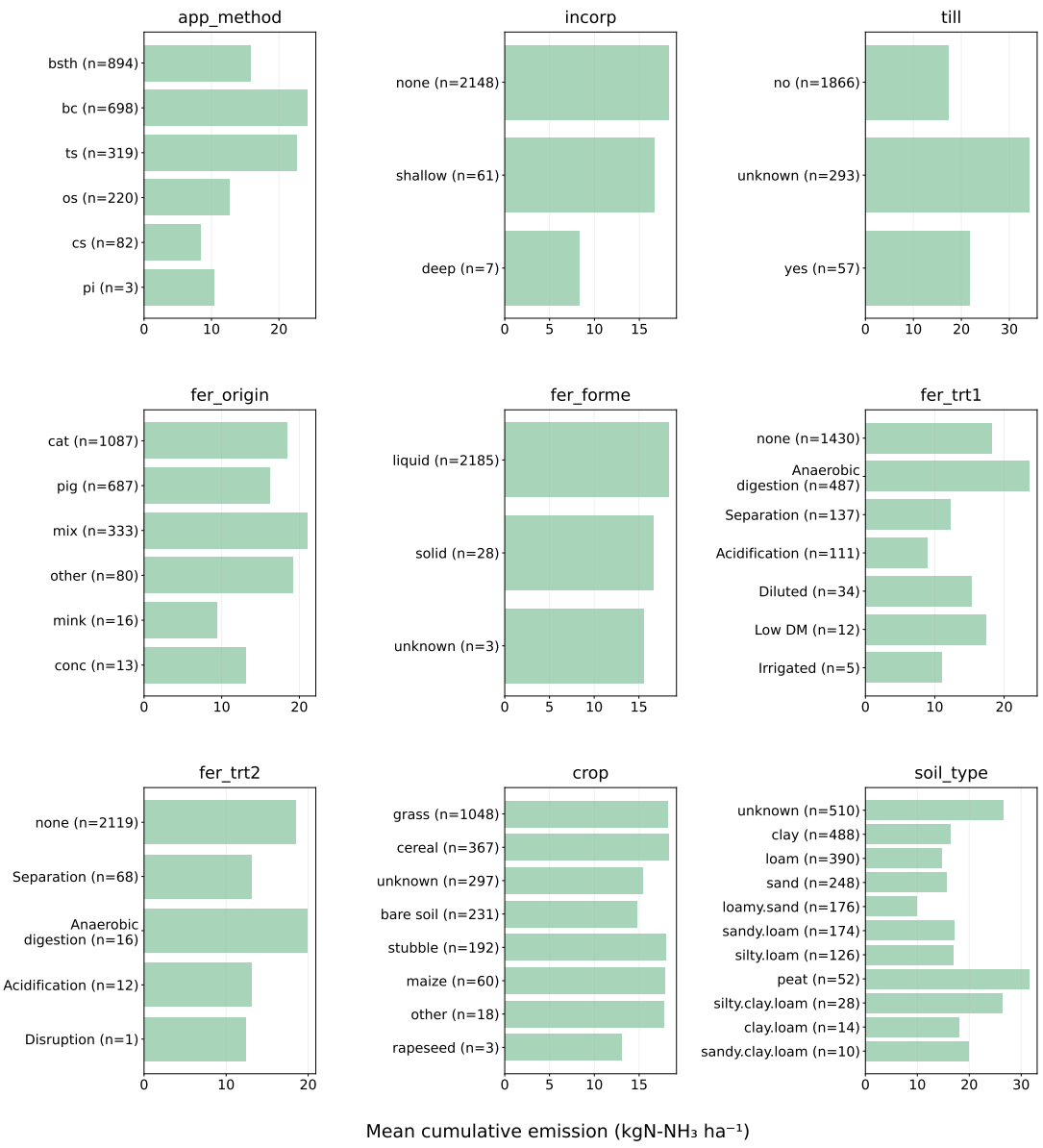


Figure 4: Partial dependence plot of categorical predictors based on cumulative emissions. Bars show mean predictions for each category, and numbers next to category labels indicate their observed frequencies in the interpretation dataset.

For every plot i whose observed value belongs to interval k, predictor j is successively set to the lower and upper limits of that interval while all other predictors, denoted by $x_{i,-j}$, are retained. The local change in predicted final emission is

$$\Delta_{ik} = f(z_k, x_{i,-j}) - f(z_{k-1}, x_{i,-j}).$$

The mean local effect within interval k is then

$$\delta_k = \frac{1}{n_k} \sum_{i:x_{ij} \in [z_{k-1}, z_k]} \Delta_{ik},$$

where n_k is the number of contributing plots in that interval. These mean local effects are accumulated across successive intervals. At boundary z_k, the uncentered accumulated effect is

$$\widetilde{ALE}_j(z_k) = \sum_{\ell=1}^{k} \delta_\ell, \qquad \widetilde{ALE}_j(z_0) = 0.$$

Finally, the curve is centered by subtracting the weighted average of the accumulated effect at the midpoint of each interval:

$$C_j = \frac{\sum_{k=1}^{K} n_k \left[\widetilde{ALE}_j(z_{k-1}) + \widetilde{ALE}_j(z_k) \right] / 2}{\sum_{k=1}^{K} n_k},$$

$$ALE_j(z_k) = \widetilde{ALE}_j(z_k) - C_j.$$

An ALE value of +2 kgN-NH_3 ha^{-1} is therefore not an absolute prediction. It indicates that the model effect around that part of the predictor distribution is approximately 2 kgN-NH_3 ha^{-1} above its average centered effect. The shape and slope of the curve are consequently more informative than its absolute vertical position: an increasing curve indicates that locally increasing the predictor tends to increase the predicted final emission, while a decreasing curve indicates the opposite.